INTERVIEW PROJECT GUIDE

ProjectHub Management System

A full-stack project and task management application built with React, TypeScript, Express, Supabase, and Tailwind CSS.

> **One-sentence explanation:** ProjectHub lets authenticated users create projects, organize tasks inside them, track progress, and view a dashboard backed by Supabase data and row-level security.

Prepared from the implementation in the ProjectHub repository
Suggested use: a 5-10 minute technical project walkthrough

1. What Problem It Solves

ProjectHub gives an individual user one place to manage projects and the work belonging to each project. Instead of keeping project status, task priority, due dates, and completion progress in separate places, the application connects them in one workflow.

Core capabilities

- Email/password registration and sign-in
- Project creation, editing, filtering, and deletion
- Task creation, editing, deletion, and status changes
- Dashboard statistics and completion progress

Technical goals

- Type-safe frontend models
- Reusable form, modal, badge, and spinner components
- Owner-scoped database access
- Deployable frontend and backend

2. Technology Stack

Layer	Technology	Role
Frontend	React 18, TypeScript, Vite	Component-based UI, routing, and fast development/build tooling.
Styling	Tailwind CSS	Utility-based responsive layout and visual states.
Icons	Lucide React	Consistent interface icons.
Authentication	Supabase Auth	Email/password registration, sessions, and sign-out.
Database	Supabase PostgreSQL	Projects, tasks, relationships, constraints, and timestamps.
Backend	Node.js, Express, CORS, dotenv	Health endpoint, project/task REST endpoints, and production static hosting.
Deployment	Vercel, Render, GitHub Actions	Frontend deployment and backend deployment automation.

3. Frontend: Where It Starts and Ends

```
Browser loads index.html
       |
       v
src/main.tsx -> createRoot(...).render(<App />)
       |
       v
src/App.tsx -> BrowserRouter + AuthProvider
       |
       +--> no session: Login/Register
       |
       +--> session: ProtectedRoutes -> AppLayout
                              |
                              +--> Dashboard
```

```
                                +--> Projects
                                +--> Project Detail
```

Frontend startup

The frontend starts with `npm run dev`, which runs Vite. The browser loads `index.html`, and `src/main.tsx` mounts the React application into the root DOM element.

`App.tsx` wraps the UI in `BrowserRouter` and `AuthProvider`. The authentication provider first restores the saved Supabase session, then listens for future authentication changes.

Protected navigation

While authentication is loading, the app shows a full-page spinner. If there is no user, private routes redirect to `/login`. If a user exists, the shared `AppLayout` provides navigation, user information, and sign-out.

Frontend ending point

A user interaction ends when the component validates input, calls the data API, receives a response, updates React state, and re-renders the screen. A production build ends at the `dist/` folder, which can be hosted as static files.

4. Main User Workflow

1. **Register or sign in.** The Auth page validates the form and calls Supabase Auth.
2. **Open the dashboard.** The app loads projects and tasks and calculates totals, pending work, completed work, and completion rate.
3. **Create a project.** The user enters a name, description, status, and optional dates. Client validation prevents an empty name and an end date before the start date.
4. **Open project details.** The project page loads the project and only its related tasks.
5. **Create and manage tasks.** Each task has a name, description, priority, status, due date, and project relationship.
6. **Track progress.** Clicking a task status cycles it from pending to in progress to completed. The project progress bar is recalculated from completed tasks.
7. **Search and filter.** Projects can be filtered by name and status. Tasks can be filtered by name, status, and priority.
8. **Edit or delete.** Users can update records or delete a task/project after confirmation. Deleting a project cascades to its tasks.
9. **Sign out.** Supabase clears the session and the app returns to the login screen.

5. Backend: Where It Starts and Ends

```
server/index.js
    |
    +--> dotenv loads environment variables
    +--> Express creates the HTTP app
    +--> CORS and JSON middleware run
    +--> Supabase client is created
    +--> routes are registered
    +--> dist/ is served for production
    +--> app.listen(PORT)

HTTP request -> route handler -> Supabase query -> JSON/HTTP response
```

Backend startup

The backend starts with `npm start` or `npm run dev` inside the `server/` folder. It loads `SUPABASE_URL` and `SUPABASE_SERVICE_KEY`, creates a Supabase client, registers middleware and routes, and listens on `PORT` or port 3000.

REST endpoints

Method	Endpoint	Purpose
GET	`/health`	Returns an availability check.
GET	`/projects`	Returns projects ordered by creation date.
GET	`/projects/:id`	Returns one project.
GET	`/tasks`	Returns tasks with project information.
GET	`/projects/:id/tasks`	Returns tasks for one project.
POST	`/tasks`	Creates a task from the JSON request body.
PUT	`/tasks/:id`	Updates a task.
DELETE	`/tasks/:id`	Deletes a task and returns HTTP 204.

Backend ending point

Each request ends when Express sends a status and response body. In production, unmatched routes fall back to `dist/index.html`, allowing the React router to handle browser routes.

> **Architecture detail to explain honestly:** The current React data functions in `src/lib/api.ts` call Supabase directly. The Express API is implemented as a parallel REST layer and is not currently the path used by those frontend functions.

6. Database and Security

The migration creates two related tables:

Table	Important fields	Relationship
`projects`	id, name, description, status, dates, user_id, created_at	Owned by an authenticated user.
`tasks`	id, name, description, priority, status, due_date, project_id, user_id, created_at	Belongs to a project; project deletion cascades.

PostgreSQL constraints restrict valid project statuses, task statuses, and task priorities. Row-level security policies allow authenticated users to select, insert, update, and delete only rows whose `user_id` matches `auth.uid()`.

7. A Strong Technical Explanation

"I built ProjectHub as a full-stack project management application. The frontend is a Vite React TypeScript app. It starts in `main.tsx`, restores authentication through Supabase, and uses React Router to separate public auth pages from protected dashboard pages. The dashboard and project detail pages use a small typed API module to read and mutate projects and tasks in Supabase. The database has owner-scoped row-level security, so a signed-in user can only work with their own data. I also added an Express backend with health, project, and task endpoints, and it can serve the compiled Vite frontend in production. The key workflow is sign in, create a project, add tasks, update task status, and use the dashboard to measure progress."

8. Engineering Decisions and Tradeoffs

Why TypeScript?

Project and task states are represented by explicit interfaces and union types. This reduces invalid status/priority values and improves refactoring safety.

Why Supabase?

It provides authentication, PostgreSQL, generated timestamps, and row-level security without requiring a custom identity system.

Why reusable UI components?

Buttons, forms, modals, spinners, and badges keep the pages consistent and reduce duplicated UI behavior.

Why client-side filtering?

The project list and loaded task list are small, so filtering in React gives immediate feedback without extra database requests.

Future improvements worth mentioning

- Route frontend mutations through the Express API consistently, or remove the unused duplicate layer.
- Add authenticated middleware to the Express routes before exposing them publicly.
- Add automated tests for auth redirects, project CRUD, task status transitions, and RLS behavior.
- Add pagination or server-side filtering for larger datasets.

- Improve error reporting for failed dashboard task deletion and add optimistic updates where appropriate.

9. Running the Project

Purpose	Command	Result
Frontend development	`npm install` `npm run dev`	Starts Vite development server.
Frontend validation	`npm run lint` `npm run typecheck`	Checks style and TypeScript types.
Frontend production build	`npm run build`	Creates `dist/`.

Backend development	`cd server`
	`npm install`
	`npm run dev`
		Starts Express with nodemon.

Backend production	`cd server` `npm start`	Starts Node and serves API/static files.

Required configuration includes frontend Supabase variables such as `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`. The backend uses `SUPABASE_URL` and `SUPABASE_SERVICE_KEY`.

ProjectHub interview guide | Generated from the Project Management System source tree